DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Student Name:	DADDANALA RANGA MANOJ KUMAR REDDY	Roll / Reg No:	252U2R5033
Course	ADBMS	Section / Room:	AI AND DE 2A
Experiment No:	Lab report 3	Date of Execution:	08/11/2026

ADBMS — Lab Report

Github repo link : https://github.com/manojdaddanala50-eng/Backend-Development-Fundamentals

Experiment 3: REST API, Query Parameters, Validation & Debugging

1. AIM & OBJECTIVES

- Refactor basic database CRUD routes into a clean, stateless RESTful API adhering to noun-based URIs (/books).
- Implement dynamic query parameters on GET /books: search_q (LIKE), order_by, order, limit, offset, and price range.
- Enforce strict request body validation on POST /books, returning 400 Bad Request with descriptive JSON for errors.
- Apply correct HTTP status codes: 200 OK, 201 Created, 400 Bad Request, and 404 Not Found.
- Secure all queries against SQL injection using parameterized placeholders (?).
- Practice backend debugging by solving port conflicts (EADDRINUSE), import paths, typos, and missing await calls.

2. RELATIONAL DATABASE SCHEMA (SQLite3)

```
CREATE TABLE IF NOT EXISTS book (
  id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT NOT NULL, author TEXT NOT NULL,
  price INTEGER NOT NULL, pages INTEGER NOT NULL );
```

Sample Catalog Records (10 Seed Rows):

ID	Book Title	Author	Price (Rs)	Pages
1	Harry Potter and the Sorcerer's Stone	J.K. Rowling	450	320
2	Clean Code	Robert C. Martin	650	464
3	JavaScript: The Good Parts	Douglas Crockford	350	176
4	Design Patterns: Elements of Reusable Software	Erich Gamma	800	395
5	You Don't Know JS: Scope & Closures	Kyle Simpson	550	278
6	The Pragmatic Programmer	Andrew Hunt	700	352
7	Eloquent JavaScript	Marijn Haverbeke	400	472
8	Introduction to Algorithms	Thomas H. Cormen	950	1312
9	Refactoring: Improving Existing Code	Martin Fowler	600	448
10	Designing Data-Intensive Applications	Martin Kleppmann	850	616

3. REST API ENDPOINTS SPECIFICATION

Method	Route Path	Key Parameters / Payload	Status	Description
GET	`/books`	search_q, order_by, order, limit, offset, min/max_price	200 OK	Search, sort, filter & paginate
GET	`/books/:bookId`	bookId (path parameter)	200 / 404	Retrieve single book by ID
POST	`/books`	{ title, author, price, pages } (body)	201 / 400	Create book with validation
PUT	`/books/:bookId`	bookId + updated fields (body)	200 / 404	Update book by ID
DELETE	`/books/:bookId`	bookId (path parameter)	200 / 404	Remove book by ID

4. CORE IMPLEMENTATION HIGHLIGHTS (server.js)

```
// GET /books with search, sorting whitelist, and price range filtering
app.get("/books", async (req, res) => {
  const { search_q = "", order_by = "id", order = "ASC", limit = 10, offset = 0,
          min_price = 0, max_price = 1000000 } = req.query;
  const allowedCols = ["id", "title", "author", "price", "pages"];
  const sortCol = allowedCols.includes(order_by) ? order_by : "id";
  const sortOrder = order.toUpperCase() === "DESC" ? "DESC" : "ASC";
  const query = `SELECT * FROM book WHERE title LIKE ? AND price BETWEEN ? AND ?
                 ORDER BY ${sortCol} ${sortOrder} LIMIT ? OFFSET ?;`;
  const books = await db.all(query, [`%${search_q}%`, min_price, max_price, limit, offset]);
  res.status(200).json(books);
});

// POST /books with strict input validation
app.post("/books", async (req, res) => {
  const { title, author, price, pages } = req.body;
  if (!title || typeof title !== "string" || !title.trim()) {
    return res.status(400).json({ message: "Validation Error: 'title' is required." });
  }
  if (!author || typeof author !== "string" || !author.trim()) {
    return res.status(400).json({ message: "Validation Error: 'author' is required." });
  }
  if (price === undefined || isNaN(Number(price)) || Number(price) < 0) {
    return res.status(400).json({ message: "Validation Error: 'price' must be positive number." });
  }
  const result = await db.run(`INSERT INTO book (title, author, price, pages) VALUES (?, ?, ?, ?)`,
                              [title.trim(), author.trim(), Number(price), Number(pages) || 100]);
  res.status(201).json({ message: "Book Added Successfully", bookId: result.lastID });
});
```

5. DEBUGGING EXERCISE: FAULTS IDENTIFIED & RESOLVED

Bug 1: Port Conflict (EADDRINUSE)

Cause: A previous node process remained listening on port 3000.
Fix: Added dynamic port fallback (const PORT = process.env.PORT || 3000) and killed the PID via netstat/Stop-Process.

Bug 2: Wrong Import Path

Cause: Relative module resolution failed when importing the SQLite database handle.
Fix: Destructured { open } directly from "sqlite" and used path.join(__dirname, "books.db") for absolute safety.

Bug 3: Table-Name Typo (no such table: books)

Cause: The database DDL created singular table "book", while endpoints queried plural "books".
Fix: Synchronized and unified all SQL statements to reference singular table "book" consistently.

Bug 4: Missing "await" Keyword on Database Calls

Cause: Calling db.all/db.run without await returned pending Promises, sending empty JSON prematurely.
Fix: Declared route handlers as async (req, res) and placed await before every database operation.

6. VERIFICATION TEST RESULTS (Automated & Postman Suite)

Test Scenario	Endpoint & Parameters	Expected	Observed Response	Result
Search Filter	`GET /books?search_q=Clean`	200 OK	Returned "Clean Code" by Robert C. Martin	**PASS**
Sort Price DESC	`GET /books?order_by=price&order=DESC`	200 OK	Top 1: "Introduction to Algorithms" (Rs 950)	**PASS**
Price Range	`GET /books?min_price=600&max_price=800`	200 OK	4 books returned within [600, 800]	**PASS**
Single Book	`GET /books/2`	200 OK	Returned Book with ID 2	**PASS**
Non-existent ID	`GET /books/9999`	404 Not Found	{"message": "Book with ID 9999 not found"}	**PASS**
Missing Title	`POST /books { author, price }`	400 Bad Request	{"message": "Validation Error: 'title' is required"}	**PASS**
Non-numeric Price	`POST /books { title, price: "abc" }`	400 Bad Request	{"message": "Validation Error: 'price' must be number"}	**PASS**
Create Book	`POST /books (valid fields)`	201 Created	{"message": "Book Added Successfully", "bookId": 11}	**PASS**
Update Book	`PUT /books/11 (valid fields)`	200 OK	{"message": "Book with ID 11 updated successfully"}	**PASS**
Delete Book	`DELETE /books/11`	200 OK	{"message": "Book with ID 11 deleted successfully"}	**PASS**

7. VIVA VOCE QUESTIONS & COMPREHENSIVE ANSWERS

Q1: Difference between Query Parameter and Path Parameter?

Path parameters (/books/:id) uniquely pinpoint an individual resource entity. Query parameters (/books?order=DESC) optionally filter, sort, or paginate the collection.

Q2: What does "stateless" mean in REST architecture?

The server stores no client session state between requests. Every request must supply all context, parameters, and tokens required for processing independently.

Q3: Which status code for bad input vs a missing record?

400 Bad Request indicates malformed/invalid client input. 404 Not Found indicates syntactically valid requests targeting non-existent database records.

Q4: Why use parameterized queries (?) against SQL injection?

Placeholders separate SQL syntax parsing from user data. Values are treated strictly as data literals, preventing malicious SQL code injection.

Faculty Evaluation Rubric:

• Working Output (50%) • Code Quality (20%) • Testing Evidence (15%) • Viva Voce (15%)

Faculty Signature: _____ Date: 08/11/2026